

Analyse détaillée du fichier style.css

Le CSS (Cascading Style Sheets) est le langage qui gère l'apparence. Si le HTML est le squelette, le CSS est la peau, les vêtements et le maquillage. Ce fichier est particulièrement intéressant car il utilise des techniques modernes pour garantir que le site est beau sur mobile comme sur ordinateur.

1. La Configuration Globale (:root & body)

C'est ici que l'on définit les règles du jeu pour tout le site.

A. Les Variables CSS (:root)

C'est la **palette de peintre** de votre site.

```
:root {  
  --primary-accent-color: #DAA520; /* Or / Moutarde */  
  --secondary-accent-color: #8B4513;  
  --dark-bg-color: #1C1C1C; /* Fond très sombre */  
  /* ... autres couleurs et polices ... */  
}
```

- **Architecture** : Au lieu d'écrire #DAA520 partout dans le fichier, on utilise --primary-accent-color.
- **Utilité** : Si demain vous voulez que votre site devienne bleu, vous changez **une seule ligne** ici, et tout le site change (liens, boutons, bordures). C'est essentiel pour la maintenance.

B. Le Reset Universel (*)

```
* { margin: 0; padding: 0; box-sizing: border-box; }
```

- margin: 0; padding: 0; : Enlève les espaces par défaut que les navigateurs ajoutent (comme autour de la page body). On part d'une page blanche propre.
- box-sizing: border-box; : **Crucial**. Cela change la façon dont la taille des boîtes est calculée.
 - *Sans ça* : Largeur = contenu + padding + bordure.
 - *Avec ça* : Largeur = ce qu'on a décidé (le padding est "écrasé" à l'intérieur). Cela

évite que les éléments dépassent de l'écran.

C. Le Corps de page (body)

```
body {  
  font-family: var(--font-body); /* Utilise la variable définie plus haut */  
  background-color: var(--dark-bg-color); /* Fond sombre */  
  scroll-behavior: smooth; /* Défilement fluide lors du clic sur les ancres */  
}
```

2. L'En-tête et la Navigation (.main-header)

Cette section gère la barre de menu qui reste en haut.

A. Le Header "Sticky"

```
.main-header {  
  position: sticky;  
  top: 0;  
  z-index: 1000;  
  /* ... */  
}
```

- `position: sticky; top: 0;` : Fait en sorte que le menu "colle" au plafond de la fenêtre quand on scrolle vers le bas. L'utilisateur a toujours accès au menu.
- `z-index: 1000;` : Définit l'ordre de superposition (l'axe Z). 1000 est un chiffre élevé pour s'assurer que le menu passe **au-dessus** de tout le reste (comme la carte météo).

B. Flexbox pour la mise en page (.navbar)

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

- `display: flex;` : Active le mode "Flexbox", l'outil moderne d'alignement.
- `justify-content: space-between;` : Pousse le premier élément (Logo) tout à gauche et le dernier (Menu) tout à droite. L'espace vide est mis au milieu.
- `align-items: center;` : Aligne verticalement le logo et les liens pour qu'ils soient sur la même ligne médiane.

C. L'Animation de soulignement (::after)

C'est un effet visuel avancé pour les liens du menu.

```
.nav-links a::after {  
  content: "";  
  position: absolute;  
  width: 0; /* Au départ, le trait est invisible (largeur 0) */  
  height: 2px;  
  /* ... */  
  transition: width 0.3s ease; /* Animation de la largeur */  
}
```

```
.nav-links a:hover::after {  
  width: calc(100% - 32px); /* Au survol, le trait s'étend */  
}
```

- **Architecture UI** : On crée un faux élément (::after) qui est un petit rectangle sous le texte. Au survol (:hover), on l'agrandit. C'est plus élégant qu'un simple text-decoration: underline.

3. Le Cœur de l'App Météo (.weather-container)

A. Le Conteneur Central

```
.weather-container {  
  max-width: 600px;  
  margin: 0 auto; /* Centre le bloc horizontalement */  
  /* ... ombres et bordures ... */  
}
```

- margin: 0 auto; : La technique classique pour centrer un bloc au milieu de la page.

B. Les Champs de Formulaire (input:focus)

```
.weather-form input[type="text"]:focus {  
  outline: none;  
  border-color: var(--primary-accent-color);  
  box-shadow: 0 0 0 3px rgba(218, 165, 32, 0.2);  
}
```

- `:focus` : Cible l'état quand l'utilisateur clique dans le champ pour écrire.
- **UX (Expérience Utilisateur)** : On remplace le contour bleu moche par défaut du navigateur par une lueur dorée (votre couleur d'accentuation). Cela confirme visuellement à l'utilisateur qu'il est "dans" le champ.

4. La Grille de Résultats (.weather-data-grid)

C'est ici qu'on affiche les cartes (Température, Humidité, etc.). C'est un bijou de CSS moderne.

```
.weather-data-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 15px;
}
```

- `display: grid;` : Active le système de Grille (plus puissant que Flexbox pour la 2D).
- `repeat(auto-fit, minmax(180px, 1fr))` : **Cette ligne est magique.** Elle dit :
 - "Crée autant de colonnes que possible."
 - "Chaque colonne doit faire au moins 180px de large."
 - "S'il reste de la place, partage-la équitablement (1fr)."
- **Architecture** : Grâce à cette seule ligne, le design est **responsive sans Media Queries**. Sur PC, vous aurez 4 colonnes. Sur tablette, 3 ou 2. Sur mobile, 1 seule colonne. Tout se calcule automatiquement.

Animation "Bounce"

```
@keyframes bounce {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-5px); }
}

.weather-card .emoji { animation: bounce 1s ease-in-out infinite; }
```

- Crée une petite animation où l'emoji saute de 5 pixels vers le haut (`translateY(-5px)`) et redescend en boucle (infinite). Cela donne vie à l'interface.

5. L'Autocomplétion (.autocomplete-items)

C'est la liste de suggestions de villes qui apparaît sous la barre de recherche.

```
.autocomplete-items {  
  position: absolute;  
  z-index: 99;  
  top: 100%; /* Se place juste en dessous du champ (100% de la hauteur du parent) */  
  /* ... */  
}
```

- `position: absolute;` : Sort l'élément du flux normal du document. Il "flotte" par-dessus le reste. C'est nécessaire pour que la liste de suggestions passe *par-dessus* le résultat météo ou la carte, sans pousser tout le contenu vers le bas.

6. Le Responsive Design (@media)

C'est ici qu'on gère l'adaptation spécifique aux mobiles si la grille automatique ne suffit pas.

```
@media (max-width: 992px) {  
  .nav-links ul {  
    display: none; /* On cache le menu par défaut sur mobile */  
    flex-direction: column; /* On passe de horizontal à vertical */  
    position: absolute; /* On le sort du flux pour en faire un menu déroulant */  
    /* ... */  
  }  
  .menu-toggle { display: flex; } /* On affiche le bouton Burger */  
}
```

- **Logique** : Si l'écran fait moins de 992 pixels de large (tablette/mobile), on change la stratégie. Le menu horizontal devient vertical et caché, et le bouton "hamburger" (les 3 traits) apparaît. C'est le JavaScript qui ajoutera la classe `.active` pour afficher le menu quand on clique.

7. Accessibilité (.sr-only, .skip-link)

```
.sr-only { ... clip: rect(0, 0, 0, 0); ... }
```

- **Architecture Accessible** : Cette classe cache visuellement un élément (comme le label

"Nom de la ville") mais **le laisse lisible** pour les robots et les lecteurs d'écran pour aveugles. C'est une bonne pratique majeure du web moderne.

Résumé

Ce fichier style.css n'est pas juste une liste de couleurs. Il structure l'application :

1. Il assure la **cohérence** (variables).
2. Il gère la **mise en page complexe** (Flexbox & Grid).
3. Il gère l'**interactivité** (Hover, Focus, Animations).
4. Il gère l'**adaptation** aux différents écrans (Responsive).